

面向事件驱动智能体操作系统的跨层能力治理方法

摘要

事件驱动智能体操作系统允许外部事件自主触发智能体并调用工具，但事件来源、任务创建与工具副作用分属不同信任域，现有事件格式校验或单点工具授权无法约束完整执行链。本文提出跨层能力治理方法 E2AG。该方法首先以来源-类型能力契约验证事件声明权，再依据目标智能体的来源、事件类型、动作、工具与环境策略决定任务创建；任务获准后，系统签发绑定事件、任务、智能体和工具集合的短期能力授权，并在模型产生的实际 MCP 调用到达上游前再次验证。算法同时将各阶段对象与判定写入统一执行溯源链。本文在事件驱动智能体操作系统原型上实现 E2AG，并设置无治理、仅契约、仅调度策略和仅运行时工具约束等基准方法。在包含 30 个正常事件和 30 个攻击事件的冻结矩阵上，完整方法阻断 30/30 个攻击并通过 30/30 个正常事件；仅契约和仅调度策略分别阻断 10/30 和 20/30 个攻击。480 次确定性端到端执行表明，调度前与工具调用时两个执行点缺一不可。进一步使用真实模型完成 30 次工具决策并在三种配置下形成 90 条执行路径：模型在工具升级场景中 10/10 次选择越权工具；仅调度门控时 10/10 次禁止副作用到达上游，启用运行时能力验证后上游到达数降为 0/10；完整方法同时阻断 10/10 个伪造来源事件。100 条故障注入轨迹全部正确定位治理失败阶段，并发实验验证了修复后的事件去重与审批状态不变量。结果表明，跨层能力生命周期能够同时约束事件触发权、任务创建权与工具副作用权。

关键词：人工智能操作系统；事件驱动智能体；能力治理；运行时强制；执行溯源

中图法分类号：TP309

A Cross-Layer Capability Governance Method for Event-Driven Agent Operating Systems

Abstract: Event-driven agent operating systems allow external events to autonomously trigger agents and tool calls. Event provenance, task creation, and tool side effects, however, cross distinct trust domains; event-format validation or a single tool guard cannot constrain the complete execution chain. This paper presents E2AG, a cross-layer capability-governance method. E2AG first validates an event's declaration authority through a source-type capability contract and then decides task creation against target-scoped source, event-type, action, tool, and environment policies. For each admitted task, E2AG issues a short-lived capability bound to the event, task, agent, and tool set, and validates it again before an actual MCP call reaches the upstream tool. All objects and decisions are recorded in a unified execution-provenance chain. We implement E2AG in an event-driven agent-OS prototype and compare it with no-governance, contract-only, dispatch-policy-only, and runtime-tool-only baselines. On a frozen matrix of 30 benign and 30 attack events, full E2AG blocks 30/30 attacks while admitting 30/30 benign events; contract-only and dispatch-policy-only block 10/30 and 20/30 attacks, respectively. Across 480 deterministic end-to-end executions, both enforcement points are necessary. We further obtain 30 tool decisions from a real model and replay them through three governance configurations, yielding 90 executions. The model selects the unauthorized tool in 10/10 escalation cases. All 10 forbidden effects reach the upstream under dispatch-only enforcement, whereas runtime capability validation reduces upstream reachability to 0/10; full E2AG also blocks all 10 spoofed-source events. One hundred fault-injection traces correctly identify their first governance failure stage, and concurrency tests validate the repaired deduplication and approval-state invariants. These results show that a cross-layer capability lifecycle jointly constrains event-trigger authority, task-creation authority, and tool-side-effect authority.

Key words: agent operating system; event-driven agent; capability governance; runtime enforcement; execution provenance

1 引言

智能体操作系统 (Agent OS/AIOS) 将模型、记忆、工具和任务调度组织为共享运行环境, 使智能体能够持续感知外部状态并自主执行操作 [1, 2]。Git 推送、邮件、监报告警和业务回调由此不再只是输入数据, 而成为创建智能体任务的触发条件。执行链从外部事件开始, 经任务调度、模型决策和工具调用到达数据、服务或设备; 其中任一阶段发生授权错配, 都会把不可信输入转化为真实副作用。

现有机制分别保护这条链的局部环节。CloudEvents 统一事件上下文, A2A 描述智能体任务交互, MCP 规范模型与工具之间的调用接口 [12, 13, 14]; 间接提示词注入研究表明, 不可信事件内容能够改变模型的工具选择 [6, 7, 8]; 运行时规则与能力系统则在动作执行前检查安全约束 [10, 11]。然而, 事件格式正确不代表其来源有权声明该事件类型, 智能体声明某项能力不代表当前事件有权创建任务, 而任务获准执行也不代表模型随后选择的任意工具都应被放行。三个授权问题分属事件、任务与工具对象, 单一入口校验或单一工具过滤均无法建立完整授权关系。

本文研究事件驱动 AIOS 中的跨层能力治理问题: 系统如何把事件声明权、目标任务创建权和工具副作用权组织为一个可执行的授权生命周期, 并使每次决策具有可验证的对象依赖证据? 为此, 本文提出 E2AG (Event-to-Agent Governance)。E2AG 将 Connector 的事件能力转化为来源-类型契约, 在任务创建前对目标智能体执行三态策略判定; 任务获准后, 系统签发绑定事件、任务、智能体、工具服务和允许工具集合的短期能力, 并在实际 MCP 调用到达上游前再次验证。事件、任务、能力与调用共享同一 trace, 其判定结果构成哈希链接的执行溯源。

E2AG 的关键思想不是增加更多独立规则, 而是使授权对象沿执行链逐层收敛: 事件契约限定谁能声明何种事件, 目标策略限定该事件能否创建特定智能体任务, 任务能力限定该任务能够产生的工具副作用。前一阶段的授权结果成为后一阶段能力的签发依据, 因而工具调用不能脱离触发它的事件和任务单独获得权限。

本文的主要贡献如下:

- (1) 建立事件-任务-工具三层执行模型, 形式化区分事件声明权、任务创建权和工具副作用权, 并给出四项可检查安全性质;
- (2) 提出跨层能力治理方法 E2AG, 设计来源-类型能力契约、目标作用域三态策略、任务作用域短期能力和双执行点授权算法;
- (3) 在事件驱动智能体操作系统原型中实现 E2AG, 通过单点治理基准、完整消融、480 次确定性端到端执行、90 条真实模型执行路径、100 条故障注入轨迹和并发状态实验验证方法有效性。

2 相关工作

2.1 智能体攻击与评测

间接提示词注入利用智能体读取的不可信数据混淆指令与内容, 使攻击者无需直接控制用户提示即可影响模型行为 [6]。InjecAgent 以工具集成任务评估间接注入, AgentDojo 则提供包含任务、工具和攻击的动态评测环境 [7, 8]; ToolEmu 通过模型模拟工具执行结果, 扩大长尾风险测试范围 [9]。这类研究刻画了模型介导的攻击面, 但主要观测模型是否产生危险决策。E2AG 不判断自然语言是否恶意, 而是研究危险决策在系统执行链中的可达性。

国内研究已从模型自身安全、生成内容安全、隐私泄露和智能体可信等角度形成风险分类与缓解框架 [3, 4, 5]。这些工作回答了大模型及智能体面临何种风险, E2AG 进一步回答其中一类系统控制问题: 不可信外部事件如何获得任务创建权, 以及模型生成的危险调用如何在产生真实副作用前被确定性执行点约束。

2.2 智能体运行时防护

AgentSpec 通过领域专用规则描述触发条件、谓词和执行动作，在智能体运行时拦截危险操作 [10]。CaMeL 从可信查询中提取控制流和数据流，并使用能力约束不可信数据对程序流与敏感数据外泄的影响 [11]。两者说明，安全约束必须位于模型之外的确定性系统层。E2AG 与其共享运行时强制原则，但研究对象不同：AgentSpec 约束智能体动作，CaMeL 约束模型程序的数据流与能力，E2AG 则把授权起点前移到外部事件，并把事件来源、任务身份和工具能力绑定为同一生命周期。

2.3 协议、访问控制与可验证证据

CloudEvents、A2A 与 MCP 分别定义事件、任务和工具调用对象，但协议互操作语义不自动构成跨层授权 [12, 13, 14]。PDP/PEP 分离支持在执行点实施结构化策略 [15]；操作系统权限访问控制表明，能力对象和全局不变量能够为任务访问资源提供细粒度约束 [16]。W3C Trace Context 解决跨服务标识传播，哈希链接和 in-toto 则用于验证过程记录的连续性与步骤依赖 [17, 18, 19]。E2AG 将这些思想用于事件驱动智能体链路，但其研究贡献在于跨层对象绑定与授权传播，而非重新定义上述协议或密码结构。

表 1 比较最接近的方法。现有工作分别覆盖事件描述、动作规则或工具数据流；E2AG 同时约束事件声明、任务创建和工具副作用，并为三者建立统一执行证据。

表 1: 代表性方法的治理范围比较

方法	事件来源绑定	任务创建门控	工具调用强制	跨层执行证据
CloudEvents[12]	—	—	—	—
AgentSpec[10]	—	—	✓	局部
CaMeL[11]	—	—	✓	控制/数据流
E2AG	✓	✓	✓	✓

3 问题定义

3.1 执行模型与信任边界

图 1 给出本文研究的事件驱动执行链。外部事件源生成事件对象，AIOS 将事件路由为智能体任务，智能体运行时依据事件内容选择工具，工具调用最终作用于外部资源。事件接入域、智能体执行域和工具副作用域拥有不同身份与状态空间，因此跨域传递的数据不能自动继承前一域的信任。

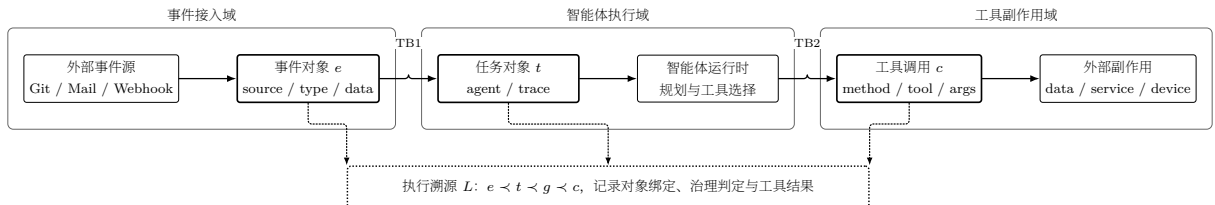


图 1: 事件驱动智能体执行模型与信任边界。粗线框表示跨层授权需要绑定的对象，点线表示执行证据写入。

定义事件

$$e = \langle id, s, \theta, d \rangle,$$

其中 s 为来源, θ 为事件类型, d 为载荷。候选目标智能体集合为 A_e 。智能体 a 的治理策略表示为

$$P_a = \langle S_a, T_a, X_a, U_a, R_a \rangle,$$

其中 S_a, T_a, X_a, U_a 分别为允许的来源、事件类型、动作和工具集合, R_a 为需要人工批准的风险条件。模型产生的工具调用为 $c = \langle m, u, q \rangle$, 分别表示 MCP 方法、工具名和参数。

3.2 攻击者能力与研究范围

攻击者能够提交或影响 Webhook、邮件、Git 内容和通用事件载荷, 能够伪造结构化字段、重放事件, 并能通过自然语言内容诱导模型改变工具选择。攻击者不能直接修改 AIOS 内部策略、治理代码或数据库。本文关注三类跨层授权错误: 未获声明权的来源伪造事件类型; 合法事件被路由到超出其能力范围的目标智能体; 任务创建后模型选择未授权工具。对已授权工具的参数级数据流攻击不属于本文算法的判定对象。

3.3 安全性质

Connector 能力契约集合为 C 。 $B_C(s, \theta)$ 判断来源 s 是否有权声明类型 θ , $D_P(e, a)$ 返回 $\{\text{allow}, \text{deny}, \text{approval}\}$ 。事件对目标 a 的接入条件为

$$\text{Admit}(e, a) = B_C(s, \theta) \wedge (D_P(e, a) = \text{allow}).$$

若策略结果为 **approval**, 只有审批状态由 **pending** 单次转移到 **approved** 后才能创建任务。

令 t 、 g 和 c 分别表示任务、任务能力与工具调用, 正常执行依赖为 $e \prec t \prec g \prec c$ 。E2AG 维护以下性质:

- I1. 任务接入性: 任务 t 的创建必须存在同一 trace 上的契约允许和策略允许, 或已完成的批准;
- I2. 能力绑定性: 能力 g 必须绑定 t 、目标智能体、MCP 服务和允许工具集合, 且具有有效期与单调状态;
- I3. 副作用授权性: 上游收到调用 c 时必须存在有效 g , 对象绑定一致且 $u(c) \in U(g)$;
- I4. 证据连续性: 同一 trace 的证据序列号连续, 且每项前驱哈希等于上一项内容哈希。

4 E2AG 跨层能力治理方法

4.1 方法概述

E2AG 在任务创建和工具调用两个不可绕过的执行点实施治理, 如图 2 所示。第一个执行点接收标准化事件, 依次验证来源-类型契约和目标策略; 第二个执行点验证任务能力, 再决定模型产生的调用能否到达上游。执行溯源不是独立日志旁路, 而是两个执行点的共同状态: 契约判定、策略结果、任务、审批、能力和工具结果均沿同一 trace 追加。

4.2 来源-类型能力契约

传统事件模式只验证字段结构。E2AG 在 Connector 描述中增加来源模式集合, 并将其与该 Connector 可产生的事件类型绑定。契约判定先检查 e 的基本结构, 再查找满足 $s \in S_k \wedge \theta \in T_k$ 的 Connector 契约 C_k 。未找到绑定时返回 **deny**; 因此, 结构合法的 `source=webhook/attacker, type=git.push` 不能借用 Git Connector 的事件语义。

契约输出包含判定、匹配契约、来源模式、策略版本和稳定原因码。这些字段既是后续目标策略的可信输入, 也是执行溯源的第一项证据。

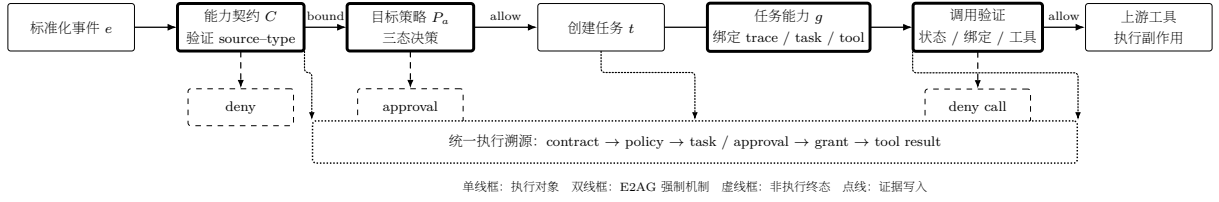


图 2: E2AG 跨层能力生命周期。调度前执行点决定事件能否创建任务，调用时执行点决定模型选择的工具能否产生副作用。

4.3 目标策略与任务能力

对每个候选目标 $a \in A_e$ ，E2AG 将事件来源、类型、声明动作、请求工具和运行环境与 P_a 比较。任一显式允许集合不匹配即返回 deny；风险条件 R_a 命中时返回 approval；其余情况返回 allow。三态决策把“需要人确认”与“策略拒绝”分开，避免高风险事件通过重试绕过审批语义。

当目标策略允许时，系统创建任务 t ，并签发

$$g = \langle trace(t), id(t), id(a), id(mcp), U_a, exp, state \rangle.$$

能力令牌只在任务运行时可见，持久化层保存其摘要。任务完成、失败或取消后， $state$ 单调转移为 revoked；超过 exp 后转移为 expired。该设计使工具授权从 Agent 级静态配置收缩到单次任务作用域。

4.4 双执行点授权算法

算法 1 给出统一处理过程。第 1–20 行完成事件接入和能力签发，第 21–30 行在 MCP 调用点验证能力。算法先完成契约判定再遍历目标策略，因此来源无权声明的事件不会创建任何目标任务；调用阶段同时检查状态、有效期、对象绑定和工具集合，任何失败均在接触上游凭据之前终止。

算法 1 E2AG 跨层能力治理算法**Require:** event e , targets A_e , contracts C , policies P , optional call c **Ensure:** admitted tasks and an allow/deny decision for each tool call

```

1: procedure GOVERN_EVENT( $e, A_e, C, P$ )
2:    $\rho \leftarrow \text{NEWTRACE}$ 
3:    $d_C \leftarrow \text{EVALUATECONTRACT}(e, C)$ ;  $\text{APPEND}(\rho, d_C)$ 
4:   if  $d_C = \text{deny}$  then
5:     return  $\emptyset$ 
6:   end if
7:    $T \leftarrow \emptyset$ 
8:   for all  $a \in A_e$  do
9:      $d_a \leftarrow \text{EVALUATETARGETPOLICY}(e, P_a)$ ;  $\text{APPEND}(\rho, d_a)$ 
10:    if  $d_a = \text{approval}$  then
11:       $d_a \leftarrow \text{CONSUMEAPPROVAL}(e, a, \rho)$ 
12:    end if
13:    if  $d_a = \text{allow}$  then
14:       $t \leftarrow \text{CREATETASK}(e, a, \rho)$ 
15:       $g \leftarrow \text{ISSUECAPABILITY}(t, a, U_a, \text{exp})$ ;  $T \leftarrow T \cup \{t\}$ 
16:       $\text{APPEND}(\rho, \langle t, g \rangle)$ 
17:    end if
18:  end for
19:  return  $T$ 
20: end procedure
21: procedure AUTHORIZEDCALL( $c, t, g$ )
22:  if  $g.\text{state} \neq \text{active} \vee \text{now} \geq g.\text{exp}$  then
23:    return deny
24:  end if
25:  if  $\neg \text{BINDINGSMATCH}(g, t, c) \vee c.\text{tool} \notin g.\text{tools}$  then
26:     $\text{APPEND}(\text{trace}(t), \langle c, \text{deny} \rangle)$ ; return deny
27:  end if
28:   $r \leftarrow \text{FORWARDTOUPSTREAM}(c)$ 
29:   $\text{APPEND}(\text{trace}(t), \langle c, \text{allow}, r \rangle)$ ; return  $r$ 
30: end procedure

```

4.5 执行溯源

每项证据 l_i 包含 sequence、stage、outcome、对象标识、前驱哈希和内容哈希。令 H 为 SHA-256, canon 为确定性 JSON 规范化, 则

$$h_i = H(\text{canon}(l_i \setminus \{h_i\})), \quad l_i.\text{previous_hash} = h_{i-1}.$$

被拒绝和待审批事件同样保留 Event 证据, 但不生成任务证据; 获准执行则继续追加 Task、Grant 和 ToolCall。由此, 验证器不仅检查链内内容是否改变, 还能依据对象标识检验 $e \prec t \prec g \prec c$ 是否成立。

5 原型实现

本文在 DiOS 事件驱动智能体操作系统原型上实现 E2AG。原型已有 Connector、CloudEvents 标准化、订阅匹配、事件调度、A2A Task 和 MCP 配置管理。E2AG 在其上增加四个协同组件：契约与策略判定器、调度前 PEP、任务能力管理器和 MCP 调用 PEP。EventLog 保存事件与治理证据，A2ATask 通过 context 和 trace 与事件关联，ToolGrant 保存任务能力摘要与生命周期状态。

调度前 PEP 位于所有 Connector 汇合到任务创建的窄腰位置，因而不需要为 Git、邮件或通用 Webhook 分别实现授权逻辑。调用 PEP 代理远程 streamable HTTP MCP：它先验证任务能力，再转发获准调用；tools/list 结果按任务允许集合裁剪，避免向任务暴露无权调用的工具。审批采用 pending、approved、rejected 和 expired 四态状态机，并以条件更新保证终态单次写入。事件去重采用带有效期的唯一声明行，消除先查后插造成的并发竞态。

实现将策略判定与 I/O 分离：纯函数负责契约和目标策略，dispatcher 负责对象创建与证据持久化，MCP PEP 负责副作用前授权。该分层使实验能够在相同执行路径上独立开关契约、目标策略和调用时 PEP。

6 实验评估

6.1 研究问题与基准方法

实验回答三个问题：

- **RQ1**：来源-类型契约与目标策略是否分别提供独立安全收益，组合后能否保持正常事件可用性？
- **RQ2**：调度前和调用时两个执行点能否共同阻止禁止副作用，真实模型工具决策是否改变结论？
- **RQ3**：执行溯源能否定位治理失败，去重与审批状态在并发条件下是否保持不变量？

表 2 给出基准方法。C 与 P 用于任务接入消融；G 与 R 分别表示调度前 PEP 和运行时 MCP PEP。ContractGuard、DispatchGuard 和 RuntimeGuard 与 E2AG 共享代码路径、数据与工具，分别实现来源-类型绑定、前置动作策略和调用时工具允许列表。它们对应事件格式/契约校验、PDP-PEP 前置门控和 AgentSpec 类运行时规则所代表的三类单点治理思路。本文比较治理位置及其组合效应，以控制智能体框架、模型和任务数据带来的混杂变量；这些基准不等价于对 AgentSpec 或 CaMeL 原实现的复现。

表 2: 实验基准方法与治理能力

编号	名称	开启机制	主要用途
C0P0 / G0R0	NoGuard	无跨层治理	无治理参照
C1P0	ContractGuard	来源-类型契约	事件声明权基准
C0P1 / G1R0	DispatchGuard	目标策略/调度前 PEP	任务创建权基准
G0R1	RuntimeGuard	任务工具 allow-list	工具副作用权基准
C1P1 / G1R1	E2AG	接入策略与运行时能力	完整方法

6.2 数据、协议与指标

RQ1 使用冻结威胁矩阵，包含 30 个正常事件和 30 个攻击事件。攻击由 10 个来源-类型绑定攻击、16 个目标来源/类型/动作/工具越权和 4 个高风险审批事件组成，覆盖 GitHub、GitLab、Gitea、IMAP、通用 Webhook、Manual 和 Cron。所有样本满足基本 CloudEvent 结构，以避免把格式错误误计为能力

治理收益。报告攻击阻断率、正常通过率和 Wilson 95% 置信区间。

RQ2 包含两组实验。确定性实验对正常授权、伪造来源、任务后工具升级和生产敏感动作, 在 G0R0、G1R0、G0R1、G1R1 下各重复 30 次, 共 480 条真实 dispatcher-SQLite-A2ATask-ToolGrant-MCP PEP 路径。真实模型实验固定三个合成事件提示, 每个重复 10 次, 使用 openai/gpt-4o-mini 生成 30 次工具决策; 每个不可变模型结果在 G1R0、G0R1、G1R1 下复用, 形成 90 条执行路径。上游为真实 loopback HTTP MCP canary。真实模型实验共使用 4900 个输入 token 和 600 个输出 token。两组均观测任务、能力、MCP 状态码、上游到达和禁止副作用, 而非只统计策略函数返回值。

RQ3 对契约拒绝、目标策略拒绝、审批过期、任务能力过期和 MCP 工具拒绝各注入 20 次, 共 100 条持久化轨迹。并发实验分别以 8 和 32 并发执行相同事件 replay 以及同一审批 approve/reject, 每种配置运行 100 轮。指标为失败阶段定位、链验证、阶段完整性、重复对象和状态不变量违规数。

6.3 RQ1: 任务接入治理的有效性

表 3: 来源-类型契约与目标策略的完整消融

配置	阻断攻击	攻击阻断率	正常通过	正常通过率
C0P0	0/30	0.00%	30/30	100.00%
C1P0	10/30	33.33%	30/30	100.00%
C0P1	20/30	66.67%	30/30	100.00%
C1P1	30/30	100.00%	30/30	100.00%

表 3 显示, ContractGuard 只阻断 10 个来源-类型绑定攻击, 其攻击阻断率置信区间为 [19.23%, 51.22%]; 当事件来源具有合法契约但目标智能体不允许该来源、动作或工具时, 契约不产生作用。DispatchGuard 阻断 16 个目标能力越权, 并将 4 个高风险动作转入审批, 阻断率区间为 [48.78%, 80.77%]; 但它缺少可信的来源-类型绑定, 因此无法识别借用合法事件类型的伪造来源。C1P1 将两类互不重叠的拒绝原因合并, 阻断 30/30 个攻击, 区间为 [88.65%, 100%]。四组均通过 30/30 个正常事件, 说明性能差异来自治理覆盖范围, 而不是普遍收紧事件接入。

6.4 RQ2: 双执行点与禁止副作用

表 4: 480 次确定性端到端执行中的上游副作用

配置	正常到达	伪造来源	工具升级	敏感动作
G0R0	30/30	30/30	30/30	30/30
G1R0	30/30	0/30	30/30	0/30 (审批)
G0R1	30/30	30/30	0/30	0/30
G1R1	30/30	0/30	0/30	0/30 (审批)

确定性结果见表 4。DispatchGuard 在任务创建前掌握事件来源和环境, 因此能够阻断伪造来源并触发敏感动作审批; 任务创建后, 它不再约束模型的实际工具选择, 故 30/30 个升级调用到达上游。RuntimeGuard 对任务能力中的工具集合执行最终检查, 阻断全部工具升级, 却无法追溯一个获准工具是否由伪造来源触发。完整 E2AG 保留 30/30 个正常副作用, 同时使三类禁止副作用均为 0/30。两个执行点的差异来自可见上下文不同, 而不是重复执行相同规则。

表 5: 真实模型决策下的 90 条执行路径

配置	正常上游到达	工具升级禁止副作用	伪造来源禁止副作用
G1R0	10/10	10/10	0/10
G0R1	10/10	0/10	10/10
G1R1	10/10	0/10	0/10

真实模型在 10 个正常事件中均选择 `ledger.record_push`, 在 10 个工具升级事件中均受载荷指令影响而选择 `secret.read`, 在 10 个伪造来源事件中均选择表面合法的 `ledger` 工具。表 5 与确定性结果一致: G1R0 无法阻止任务后的工具升级, G0R1 无法识别工具合法但来源伪造的事件, G1R1 同时阻断两类副作用。该实验验证了模型生成调用进入真实 MCP PEP 时的系统行为; 其结论是副作用可达性, 而不是模型对提示词注入的总体易感率。

6.5 RQ3: 执行溯源与并发状态

表 6: 故障注入的阶段定位与证据验证

故障阶段	轨迹数	正确定位	链有效	阶段完整
契约拒绝	20	20	20	20
目标策略拒绝	20	20	20	20
审批过期	20	20	20	20
任务能力过期	20	20	20	20
MCP 工具拒绝	20	20	20	20

100/100 条轨迹均保持统一 `trace`, 包含到达该终态所需的阶段, 并将首个治理失败定位到注入位置。原因在于 E2AG 记录的是具有终态语义的执行事件, 而非事后根据文本日志推断失败来源。对内容修改、换序、`trace` 替换、前驱哈希替换、插入和中间删除的 6 类链内篡改, 验证器均拒绝该链。

并发实验首先发现原有先查后插去重在 8 并发下的 100 轮均产生重复 `EventLog`, 共创建 588 条记录。引入数据库唯一且带有效期的去重声明后, 8/32 并发 `replay` 的 100 轮分别只创建 100 条记录; 8/32 并发 `approve/reject` 的 100 轮分别只有 100 个成功终态。四组不变量违规数均为 0。该结果说明, 授权生命周期的正确性不仅取决于判定逻辑, 也取决于事件声明和审批状态的原子转移。

7 讨论

方法边界。E2AG 将不可信事件限制在获准来源、目标和工具集合内, 不承担自然语言攻击分类。若攻击使用已授权工具和合法工具名, 仅在参数中构造危险数据, 当前工具粒度能力无法区分; CaMeL 的数据流能力或参数谓词可与 E2AG 组合, 进一步约束调用参数和敏感数据流。

内部有效性。冻结矩阵按治理层构造, 因此适合验证机制分工, 不用于估计真实攻击流量中的发生率。真实模型实验使用固定提示和强制工具选择, 保证每次运行都形成可比较的工具决策; 它证明模型调用经过双执行点后的副作用可达性, 不把 10/10 解释为通用攻击成功率。

外部有效性。当前实现和并发实验基于单一 AIOS 原型与 SQLite。方法依赖两个系统窄腰: 所有事件在任务创建前汇合, 所有远程工具副作用在调用前可被中介。具备等价窄腰的其他系统可以复用算法, 但数据库状态机和非 MCP 工具传输仍需分别验证。

证据强度。哈希链验证已取得记录的内容与顺序,不能单独证明数据库未被整体回滚或截断。外部透明日志或周期性 Merkle 根锚定可将 I4 扩展为跨存储完整性;该增强不改变 I1–I3 的执行授权语义。

8 结论

本文提出面向事件驱动智能体操作系统的跨层能力治理方法 E2AG。该方法以来源–类型契约建立事件声明权,以目标策略控制任务创建,以任务作用域短期能力控制实际工具副作用,并用统一执行溯源关联事件、任务、能力和调用。完整消融表明契约与目标策略分别覆盖不同攻击层;确定性和真实模型端到端实验共同证明,调度前与调用时执行点具有不可替代的上下文,只有二者组合才能同时阻断伪造来源和任务后工具升级;故障注入与并发实验进一步验证了对象依赖证据和状态不变量。E2AG 将分散的事件校验与工具授权转化为可执行的跨层能力生命周期,为 AIOS 中外部事件驱动的自主执行提供了系统化治理基础。

参考文献

- [1] Mei K, Zhu X, Xu W, et al. AIOS: LLM Agent Operating System. arXiv:2403.16971, 2024.
- [2] Wang L, Ma C, Feng X, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 2024, 18: 186345. [doi: 10.1007/s11704-024-40231-1]
- [3] Huang HY, Li SL, Lan TW, et al. A survey on the safety of large language model: Classification, evaluation, attribution, mitigation and prospect. *CAAI Transactions on Intelligent Systems*, 2025, 20(1): 2–32 (in Chinese with English abstract). [doi: 10.11992/tis.202401006]
- [4] Mu YY, Chen HX, Li HW. Advances in security and privacy-preserving techniques for large language models. *Journal of Cybersecurity*, 2024, 2(1): 40–49 (in Chinese with English abstract). [doi: 10.20172/j.issn.2097-3136.240103]
- [5] Zhang X, Li CZ, Xu N, Zhang LT. Security challenges and response mechanisms for trustworthy large language model agents. *Information and Communications Technology and Policy*, 2025, 51(1): 33–37 (in Chinese with English abstract). [doi: 10.12267/j.issn.2096-5931.2025.01.005]
- [6] Greshake K, Abdelnabi S, Mishra S, et al. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173, 2023.
- [7] Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: *Proc. of the Findings of ACL 2024*. Bangkok: Association for Computational Linguistics, 2024. 10471–10506. [doi: 10.18653/v1/2024.findings-acl.624]
- [8] Debenedetti E, Zhang J, Balunović M, et al. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Advances in Neural Information Processing Systems*, 2024, 37: 82895–82920. [doi: 10.52202/079017-2636]
- [9] Ruan Y, Dong H, Wang A, et al. Identifying the risks of LM agents with an LM-emulated sandbox. In: *Proc. of the 12th Int'l Conf. on Learning Representations*. Vienna: OpenReview, 2024.
- [10] Wang H, Poskitt CM, Sun J. AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents. In: *Proc. of the 48th IEEE/ACM Int'l Conf. on Software Engineering*. New York: ACM, 2026. 12 pages. [doi: 10.1145/3744916.3764546]
- [11] Debenedetti E, Shumailov I, Fan T, et al. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025.
- [12] Cloud Native Computing Foundation. CloudEvents Specification, Version 1.0.2, 2022. <https://github.com/cloudevents/spec/tree/ce/v1.0.2>. [2026-08-14]
- [13] A2A Protocol Working Group. Agent2Agent Protocol Specification. <https://a2a-protocol.org/latest/>. [2026-08-14]
- [14] Model Context Protocol Contributors. Model Context Protocol Specification, Revision 2025-06-18. <https://modelcontextprotocol.io/specification/2025-06-18/>. [2026-08-14]
- [15] Open Policy Agent. OPA Documentation: Policy Decision and Enforcement. <https://www.openpolicyagent.org/docs>. [2026-08-14]
- [16] Xu JL, Wang SL, Li LM, et al. Formal verification of capability-based access control in operating system kernel. *Journal of Software*, 2025, 36(8): 3570–3586 (in Chinese with English abstract). [doi: 10.13328/j.cnki.jos.007351]
- [17] W3C. Trace Context. W3C Recommendation, 2021. <https://www.w3.org/TR/trace-context/>.
- [18] Haber S, Stornetta WS. How to time-stamp a digital document. *Journal of Cryptology*, 1991, 3(2): 99–111. [doi: 10.1007/BF00196791]
- [19] Torres-Arias S, Afzali H, Kuppusamy TK, et al. in-toto: Providing farm-to-table guarantees for bits and bytes. In: *Proc. of the 28th USENIX Security Symp*. Santa Clara: USENIX Association, 2019. 1393–1410.